SkillNavora

STAGE 2 HANDBOOK

Stage 2: Applied AI

Outcome: Build practical LLM-powered applications. Topics: OpenAI API, Claude API (Anthropic), Gemini API (Google), Prompt engineering, Context engineering, Embeddings, Tokenisation, Basic RAG.

From Non-Coder to Enterprise AI Builder: Learn, Build, Deploy, Get Hired and Launch Your AI Business.

Varun Kumar · SkillNavora · skillnavora.in

Edition: November 2026. Links point to official documentation where available; check them for the latest versions.

Financial figures are hypothetical examples, not advice.

Stage 2: Applied AI

Outcome: Build practical LLM-powered applications

This handbook covers each topic of Stage 2 with a definition, why it matters, how it works, step-by-step instructions, example code or templates, common mistakes, interview questions with answers, a hands-on exercise and official resources.

Topics in this stage

1. OpenAI API
2. Claude API (Anthropic)
3. Gemini API (Google)
4. Prompt engineering
5. Context engineering
6. Embeddings
7. Tokenisation
8. Basic RAG

STAGE 2 · TOPIC 1

OpenAI API

DEFINITION

> The OpenAI API gives programmatic access to OpenAI's GPT models for text, reasoning, vision, audio, embeddings and tool calling. The current main endpoint is the Responses API; the older Chat Completions API is still widely used in existing code.

WHY IT MATTERS

> Many enterprise customers already use OpenAI models, often through Azure OpenAI. An FDE must be able to call the API, control output format, use tools, handle errors and estimate cost.

HOW IT WORKS

- Create an API key in the OpenAI platform and keep it in the OPENAI_API_KEY environment variable.
- Install the SDK (pip install openai) and create a client: client = OpenAI().
- Send input (text or a list of messages with roles) and a model name; read the text from the response.
- Use instructions for system-level rules, tools for function calling, and structured outputs (JSON schema) for reliable data.
- Usage fields report input and output tokens; cost = tokens × the published price per million tokens.
- Choose a model from the official models page; names change over time.

STEP-BY-STEP

1. Create an account and API key; set a monthly spending limit.
2. pip install openai; export OPENAI_API_KEY=... (never commit it).
3. Run the basic example below and print the answer and usage.
4. Add an instruction that sets tone and format.
5. Ask for JSON output and validate it with json.loads or Pydantic.

6. Wrap the call with try/except for rate limits and API errors.

EXAMPLE CODE / TEMPLATE

```
# pip install openai
from openai import OpenAI

client = OpenAI()  # reads OPENAI_API_KEY from the environment
MODEL = 'MODEL_ID'  # pick a current model from https://platform.openai.com/docs/models

resp = client.responses.create(
    model=MODEL,
    instructions='You are a concise real-estate assistant. Answer in 3 bullet points.',
    input='What should a first-time buyer check before booking a flat?',
)
print(resp.output_text)
print(resp.usage)  # input/output token counts for cost tracking
```

COMMON MISTAKES

- Hard-coding the key in source code or frontend code.
- Not setting a spending limit during learning.
- Parsing free text when you need structured data; use structured outputs.
- Assuming a model name from an old tutorial still exists.

INTERVIEW QUESTIONS AND ANSWERS

Q. How do you control cost when calling an LLM API?

A. Pick the smallest model that meets quality on your evaluation set, keep prompts and context short, cap output length, cache repeated prompts or answers, batch offline jobs, and monitor tokens per request.

Q. Responses API vs Chat Completions?

A. Both generate model output. Responses is OpenAI's newer, unified interface with built-in tools and state options; Chat Completions takes a messages list and is common in older code. Check the current docs for which features each supports.

Q. How do you get reliable JSON from a model?

A. Use the provider's structured-output feature with a JSON schema, validate the result in code, and retry or fall back when validation fails.

HANDS-ON EXERCISE

> Build a command-line tool that takes a property description and returns JSON with price_band, highlights and risks, validated with Pydantic.

OFFICIAL AND RECOMMENDED RESOURCES

- OpenAI API docs (official) https://platform.openai.com/docs
- OpenAI: text generation guide https://platform.openai.com/docs/guides/text
- OpenAI: structured outputs https://platform.openai.com/docs/guides/structured-outputs

Claude API (Anthropic)

DEFINITION

The Claude API gives access to Anthropic's Claude models through the Messages API. Claude is used for long documents, coding, analysis and agentic tool use. Current models include Claude Opus 5.5 (claude-opus-5-5, the default), Claude Sonnet 5.5 (claude-sonnet-5-5) and Claude Haiku 5.5 (claude-haiku-5-5).

WHY IT MATTERS

Claude is widely used in enterprise for document-heavy and agent workloads, and is available directly, on Amazon Bedrock, Google Cloud Vertex AI and Microsoft Foundry, which matters when customers require their own cloud.

HOW IT WORKS

- Install the SDK (pip install anthropic); the client reads ANTHROPIC_API_KEY from the environment.
- Every call goes to messages.create with model, max_tokens, an optional system prompt and a list of messages (user/assistant).
- The response contains content blocks (text, tool_use) and a stop_reason (end_turn, max_tokens, tool_use, refusal).
- Tools are defined with name, description and input_schema; Claude returns tool_use blocks that your code executes and answers with tool_result blocks.
- messages.count_tokens estimates input tokens before sending; usage on the response reports what was billed.
- Effort (output_config.effort: low to max) trades depth of reasoning for speed and cost; prompt caching cuts cost for repeated long prefixes.

STEP-BY-STEP

1. Create a key in the Claude Console; set a spend limit.
2. pip install anthropic; export ANTHROPIC_API_KEY=...
3. Run the basic call below.
4. Check stop_reason before using the text.
5. Add a tool and run the loop: call → execute tool → return result → repeat until end_turn.
6. Count tokens on a long document before sending it.

EXAMPLE CODE / TEMPLATE

```
# pip install anthropic
import anthropic

client = anthropic.Anthropic()  # reads ANTHROPIC_API_KEY

msg = client.messages.create(
    model='claude-opus-5-5',
    max_tokens=2000,
    system='You are a concise real-estate assistant. Answer in 3 bullet points.',
    messages=[{'role': 'user', 'content': 'What should a first-time buyer check before booking a
flat?'}],
)
```

```
if msg.stop_reason == 'refusal':
    print('Request declined')
else:
    print(''.join(b.text for b in msg.content if b.type == 'text'))
print(msg.usage.input_tokens, msg.usage.output_tokens)

# Count tokens before sending a long document
count = client.messages.count_tokens(
    model='claude-opus-5-5',
    messages=[{'role': 'user', 'content': open('policy.txt', encoding='utf-8').read()}],
)
print('input tokens:', count.input_tokens)
```

COMMON MISTAKES

- Reading msg.content[0].text without checking block type or stop_reason.
- Setting max_tokens too low, which cuts answers off (stop_reason = max_tokens).
- Dropping earlier assistant content blocks when continuing a tool-use conversation.
- Using model IDs from old tutorials; use the IDs listed in the official models page.

INTERVIEW QUESTIONS AND ANSWERS

Q. How does tool use work in the Claude API?
A. You send tool definitions with JSON schemas. When Claude needs a tool it returns stop_reason tool_use with tool_use blocks (name, input). Your code runs the tool and sends a user message with tool_result blocks referencing the tool_use IDs. You repeat until stop_reason is end_turn.

Q. How would you reduce cost for a chatbot that sends the same long policy document every call?
A. Use prompt caching on the stable prefix (system prompt and document), keep the volatile question after it, choose an appropriate model and effort, and verify cache hits in usage.

Q. A customer needs data to stay in their AWS account. Options?
A. Use Claude through Amazon Bedrock (or Claude Platform on AWS) in their account and region, with their IAM, logging and network controls.

HANDS-ON EXERCISE

> Build a document Q&A script: load a PDF policy, send it to Claude with a question, and return the answer with quoted supporting sentences.

OFFICIAL AND RECOMMENDED RESOURCES

- Claude API documentation (official) https://docs.claude.com/
- Claude: tool use overview https://docs.claude.com/en/docs/agents-and-tools/tool-use/overview
- Anthropic Python SDK https://github.com/anthropics/anthropic-sdk-python

Gemini API (Google)

DEFINITION

> The Gemini API gives access to Google's Gemini models for text, images, audio, video and long documents. Developers use it through Google AI Studio with an API key, or through Vertex AI on Google Cloud for enterprise controls.

WHY IT MATTERS

> Customers on Google Cloud and Google Workspace often prefer Gemini on Vertex AI. Gemini's multimodal and long-context features are useful for video, audio and large document workloads.

HOW IT WORKS

- Get a key in Google AI Studio (development) or use a Google Cloud project with Vertex AI (enterprise).
- Install the Google Gen AI SDK: pip install google-genai.
- Create a client and call models.generate_content with a model name and contents.
- Configure system instructions, temperature, safety settings and structured JSON output through a config object.
- Function calling and grounding with Google Search are available as tools.
- Choose a current model name from the official models page.

STEP-BY-STEP

1. Create an API key in Google AI Studio; set GEMINI_API_KEY.
2. pip install google-genai.
3. Run the basic example and print the text.
4. Add a system instruction and a JSON response schema.
5. Try a multimodal call with an image file.
6. For production on Google Cloud, switch the client to Vertex AI with your project and region.

EXAMPLE CODE / TEMPLATE

```
# pip install google-genai
from google import genai
from google.genai import types

client = genai.Client()  # reads GEMINI_API_KEY (or GOOGLE_API_KEY)
MODEL = 'MODEL_ID'  # pick a current model from https://ai.google.dev/gemini-api/docs/models

resp = client.models.generate_content(
    model=MODEL,
    contents='What should a first-time buyer check before booking a flat?',
    config=types.GenerateContentConfig(
        system_instruction='You are a concise real-estate assistant. Answer in 3 bullet points.',
    ),
)
print(resp.text)
```

COMMON MISTAKES

- Mixing the AI Studio and Vertex AI setups (keys vs Google Cloud credentials).

- Ignoring safety settings and finish reasons when output is blocked.
- Uploading sensitive customer files to a personal developer account.

INTERVIEW QUESTIONS AND ANSWERS

Q. When would you choose Vertex AI over AI Studio?
A. For enterprise use: Google Cloud IAM, VPC controls, data residency, audit logging, quotas and billing under the customer's project. AI Studio is for quick development and prototypes.

Q. Why build a multi-provider abstraction?
A. To compare quality and cost, meet customer cloud requirements, and fail over during outages without rewriting the application.

HANDS-ON EXERCISE

> Write one function ask(provider, prompt) that calls OpenAI, Claude or Gemini and returns plain text, then compare their answers and token costs on five prompts.

OFFICIAL AND RECOMMENDED RESOURCES

- Gemini API docs (official) https://ai.google.dev/gemini-api/docs
- Google Gen AI Python SDK https://googleapis.github.io/python-genai/
- Vertex AI generative AI docs https://cloud.google.com/vertex-ai/generative-ai/docs

Prompt engineering

DEFINITION

> Prompt engineering is designing the instructions, examples and format requests you give a model so it produces correct, consistent and useful output for a task.

WHY IT MATTERS

> Most quality problems in early AI deployments are prompt problems: vague instructions, missing context, no examples, no output format. Good prompts are cheap to change and often give the biggest quality gain.

HOW IT WORKS

- Be clear and direct: say the task, the audience, the constraints and the output format.
- Give context: who the user is, what the data is, what good looks like.
- Use examples (few-shot) to show the exact pattern you want.
- Separate instructions from data with tags such as <document>...</document> so the model does not confuse them.
- Ask for structured output (JSON with named fields) when code will read it.
- Test prompts against a fixed set of real cases and change one thing at a time.

STEP-BY-STEP

1. Write the task in one sentence and list what a perfect answer contains.
2. Collect 20 real inputs with expected outputs.
3. Write version 1 of the prompt with role, task, rules, format and two examples.
4. Run all 20 cases and score them.
5. Fix the most common failure, re-run, and keep the version that scores better.
6. Store prompts in files with version numbers.

EXAMPLE CODE / TEMPLATE

```
SYSTEM = '''You classify customer enquiries for a real-estate agency.
Return only JSON: {"intent": one of ["buy", "rent", "sell", "other"],
"budget_inr": number or null, "location": string or null, "urgency": "high"|"normal"}.
If a field is not stated, use null. Do not guess numbers.'''

EXAMPLES = '''<example>
<enquiry>Looking for a 2BHK to buy in Whitefield, around 90 lakhs, need it this month</enquiry>
<json>{"intent":"buy","budget_inr":9000000,"location":"Whitefield","urgency":"high"}</json>
</example>'''

def build_prompt(enquiry: str) -> str:
    return f'{EXAMPLES}\n<enquiry>{enquiry}</enquiry>'
```

COMMON MISTAKES

- Vague tasks ('summarise this') without audience, length or focus.
- Hiding rules deep inside long text.
- Never testing on real cases.

- Putting untrusted user text where it can be read as instructions.

INTERVIEW QUESTIONS AND ANSWERS

Q. How do you make outputs consistent?

A. Give explicit format rules, examples, a schema or structured-output feature, a lower temperature where available, and validate outputs in code.

Q. What is few-shot prompting and when does it help?

A. Including worked examples of input and desired output in the prompt; it helps when format or judgement is hard to describe in words.

Q. How do you know a prompt change is an improvement?

A. Run both versions on the same evaluation set and compare scores, not on one or two hand-picked examples.

HANDS-ON EXERCISE

> Create a prompt that extracts invoice number, date, vendor and total from invoice text, test it on 15 invoices and report the accuracy per field.

OFFICIAL AND RECOMMENDED RESOURCES

- Claude: prompt engineering overview https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview
- OpenAI: prompting guide https://platform.openai.com/docs/guides/prompt-engineering
- Google: prompt design strategies https://ai.google.dev/gemini-api/docs/prompting-strategies

Context engineering

DEFINITION

> Context engineering is deciding what information the model sees on each call (instructions, retrieved documents, memory, conversation history, tool results) and in what form, so it has exactly what it needs and nothing that distracts it.

WHY IT MATTERS

> As applications grow from single prompts to agents and long conversations, quality depends less on clever wording and more on giving the model the right facts at the right time within a limited, costly context window.

HOW IT WORKS

- The context window is limited and every token costs money and time.
- Retrieve just-in-time: fetch the documents, records or tool outputs needed for this step.
- Summarise or drop old history; keep key decisions and facts in a structured state or memory.
- Order matters: stable instructions first (also good for caching), then reference material, then the current question.
- Label each part clearly (tags or headings) and say how to use it ('answer only from the documents').
- For agents, keep tool outputs short and return only the fields the next step needs.

STEP-BY-STEP

1. List every piece of information a perfect answer requires.
2. Mark which parts are stable (rules), per-user (profile) and per-request (question, retrieved docs).
3. Build a function that assembles the context from those parts with clear tags.
4. Set a token budget for each part and trim to fit.
5. Log the final context for each request so you can debug bad answers.

EXAMPLE CODE / TEMPLATE

```
def build_context(rules: str, profile: dict, docs: list[dict], history: list[str], question: str) →
str:
    doc_text = '\n'.join(f"<doc source='{d['source']}'>{d['text']}</doc>" for d in docs[:4])
    recent = '\n'.join(history[-6:])  # keep only the last few turns
    return f'''<rules>{rules}</rules>
<customer>name={profile['name']}; plan={profile['plan']}; city={profile['city']}</customer>
<documents>
{doc_text}
</documents>
<recent_conversation>
{recent}
</recent_conversation>
<question>{question}</question>
Answer only from <documents>. If the answer is not there, say so and offer to connect a human.'''
```

COMMON MISTAKES

- Sending the entire conversation and all documents every time.
- Mixing untrusted document text with instructions without labels.

- Losing important early facts when truncating history.
- Not logging the context, so bad answers cannot be explained.

INTERVIEW QUESTIONS AND ANSWERS

Q. Context engineering vs prompt engineering?
A. Prompt engineering focuses on how instructions are written; context engineering decides the whole set of information placed in the window for each call, including retrieval, memory, history and tool results.

Q. How do you handle a conversation longer than the context window?
A. Summarise older turns, store key facts in structured memory, retrieve relevant history when needed, or use provider features such as compaction.

Q. Why put stable content first?
A. It lets prompt caching reuse the prefix, reducing cost and latency, and keeps instructions prominent.

HANDS-ON EXERCISE

> Instrument a chatbot to log context size per request, then cut average tokens by 40% without lowering scores on your evaluation set.

OFFICIAL AND RECOMMENDED RESOURCES

- Anthropic: effective context engineering for AI agents https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents
- Claude: prompt caching https://docs.claude.com/en/docs/build-with-claude/prompt-caching

Embeddings

DEFINITION

> An embedding is a vector (list of numbers) produced by an embedding model to represent the meaning of text, images or code. Items with similar meaning have vectors that are close together.

WHY IT MATTERS

> Embeddings power semantic search, RAG, recommendations, clustering, duplicate detection and classification. They let you find 'cancel my plan' when the document says 'end your subscription'.

HOW IT WORKS

- An embedding model reads text and outputs a fixed-length vector (for example 768 or 1,536 numbers).
- Similarity between two vectors is measured with cosine similarity (or dot product).
- To search: embed all documents once and store the vectors; embed each query and find the nearest vectors.
- Vector databases (pgvector, Pinecone, Qdrant) index vectors for fast nearest-neighbour search.
- Use the same embedding model for documents and queries; changing models means re-embedding everything.
- Providers include OpenAI, Google, Cohere, Voyage AI and open-source sentence-transformers models.

STEP-BY-STEP

1. pip install sentence-transformers numpy (free, runs locally).
2. Embed 10 FAQ answers.
3. Embed a user question and compute cosine similarity to each FAQ.
4. Return the best match and its score; test with paraphrased questions.
5. Set a minimum score below which you answer 'I don't know'.

EXAMPLE CODE / TEMPLATE

```
# pip install sentence-transformers numpy
from sentence_transformers import SentenceTransformer
import numpy as np

model = SentenceTransformer('all-MiniLM-L6-v2')  # small, free, local
faqs = [
    'To end your subscription, go to Settings > Billing > Cancel plan.',
    'Site visits can be booked from the property page or on WhatsApp.',
    'Home loan pre-approval usually takes 3 to 5 working days.',
]
doc_vecs = model.encode(faqs, normalize_embeddings=True)

def search(question: str, min_score: float = 0.35):
    q = model.encode([question], normalize_embeddings=True)[0]
    scores = doc_vecs @ q            # cosine similarity (vectors are normalised)
    best = int(np.argmax(scores))
    return (faqs[best], float(scores[best])) if scores[best] ≥ min_score else (None,
float(scores[best]))
```

```
print(search('how do I cancel my plan?'))
```

COMMON MISTAKES

- Mixing vectors from different embedding models.
- Embedding whole long documents as one vector, which blurs meaning.
- No similarity threshold, so irrelevant matches are always returned.
- Forgetting to store the source text and metadata with the vector.

INTERVIEW QUESTIONS AND ANSWERS

Q. What is cosine similarity?

A. The cosine of the angle between two vectors: 1 means same direction (very similar meaning), 0 unrelated. With normalised vectors it equals the dot product.

Q. Keyword search vs semantic search?

A. Keyword search matches exact terms (good for codes and names); semantic search matches meaning (good for paraphrases). Hybrid search combines both.

Q. How do you choose an embedding model?

A. Test retrieval quality on your own queries and documents, and weigh language support, vector size (storage and speed), price, hosting and licence.

HANDS-ON EXERCISE

> Embed 100 support articles, build a search function, and measure how often the correct article is in the top 3 for 30 test questions.

OFFICIAL AND RECOMMENDED RESOURCES

- OpenAI: embeddings guide https://platform.openai.com/docs/guides/embeddings
- Sentence Transformers documentation https://www.sbert.net/
- Google: text embeddings https://ai.google.dev/gemini-api/docs/embeddings

Tokenisation

DEFINITION

> Tokenisation is splitting text into tokens, the units a language model reads and writes. A token is often a word piece: 'unbelievable' might be 'un', 'believ', 'able'. Models have limits and prices measured in tokens.

WHY IT MATTERS

> Tokens decide cost, speed and whether your input fits the context window. Estimating tokens correctly is needed to budget projects, design chunking for RAG and avoid truncated outputs.

HOW IT WORKS

- Each model family has its own tokenizer; the same text gives different counts on different models.
- English averages roughly three-quarters of a word per token, but numbers, code, and many non-English languages (including Indian languages) can use many more tokens.
- Input tokens (your prompt and context) and output tokens (the reply) are counted and priced separately; output usually costs more.
- The context window limits input plus output tokens per request.
- max_tokens (or a similar setting) caps the output length.
- Count exactly with the provider's tool: Anthropic's count_tokens endpoint, OpenAI's tiktoken library, Gemini's count_tokens.

STEP-BY-STEP

1. Take a 1,000-word document and count its tokens with a provider tool.
2. Compare counts for English, Hindi and a JSON sample.
3. Calculate the cost of 10,000 such requests with the published prices.
4. Decide a chunk size in tokens for RAG and a max output length for replies.

EXAMPLE CODE / TEMPLATE

```
# Estimate monthly cost from token counts (prices are placeholders: use the provider's pricing page)
PRICE_IN_PER_M = 4.00    # $ per 1M input tokens
PRICE_OUT_PER_M = 20.00  # $ per 1M output tokens

def monthly_cost(requests_per_day: int, in_tokens: int, out_tokens: int, days: int = 30) → float:
    per_request = in_tokens / 1e6 * PRICE_IN_PER_M + out_tokens / 1e6 * PRICE_OUT_PER_M
    return round(per_request * requests_per_day * days, 2)

# 2,000 chats a day, 3,000 input tokens (instructions + retrieved docs), 300 output tokens
print(monthly_cost(2000, 3000, 300))   # → 1080.0

# Exact counts (Claude):
# client.messages.count_tokens(model='claude-opus-5-5', messages=[...]).input_tokens
```

COMMON MISTAKES

- Estimating cost from words instead of tokens.
- Forgetting that retrieved documents and conversation history count as input tokens on every call.
- Setting output limits so low that answers are cut off.

INTERVIEW QUESTIONS AND ANSWERS

Q. Why do non-English languages often cost more?
A. Tokenizers trained mostly on English split other scripts into more, smaller tokens, so the same meaning needs more tokens.

Q. How do you estimate the cost of an AI feature?
A. Measure average input and output tokens per request on realistic samples, multiply by the price per token and by expected volume, and add retries, embeddings and evaluation runs.

Q. What happens when input plus output exceeds the context window?
A. The request is rejected or the output is cut short; you must trim, summarise or chunk the input.

HANDS-ON EXERCISE

> Build a cost calculator sheet for an AI support bot: volume, tokens per call, model prices, cache hit rate, and monthly total for three models.

OFFICIAL AND RECOMMENDED RESOURCES

- Claude: token counting https://docs.claude.com/en/docs/build-with-claude/token-counting
- OpenAI tiktoken (GitHub) https://github.com/openai/tiktoken
- Hugging Face: tokenizers summary https://huggingface.co/docs/transformers/tokenizer_summary

STAGE 2 · TOPIC 8

Basic RAG

DEFINITION

> Retrieval-augmented generation (RAG) answers questions by first retrieving relevant passages from your documents and then asking the model to answer using only those passages, with citations.

WHY IT MATTERS

> Customers want AI that answers from their own policies, manuals, contracts and knowledge bases, which models were never trained on. RAG is the most common enterprise AI pattern and appears in most FDE interviews.

HOW IT WORKS

- Ingest: load documents and extract clean text with metadata (source, page, date).
- Chunk: split text into passages of a few hundred tokens with some overlap.
- Embed and store: create a vector for each chunk and store it with text and metadata.
- Retrieve: embed the question and fetch the top-k most similar chunks (optionally filtered by metadata).
- Generate: put the chunks in the prompt with instructions to answer only from them and cite sources.
- Evaluate: test retrieval (right chunks found?) and answers (faithful and relevant?) on a question set.

STEP-BY-STEP

1. Collect 5–10 real documents (PDF or text).
2. Extract text and split into ~500-token chunks with 50-token overlap.
3. Embed chunks and store them (a Python list for learning; pgvector or Qdrant for real use).
4. For each question, retrieve the top 4 chunks.
5. Send them to the model with grounding instructions and print the answer with sources.
6. Create 20 test questions with expected sources and measure how often the right source is retrieved.

EXAMPLE CODE / TEMPLATE

```
# Minimal RAG: local embeddings + Claude for generation
import anthropic, numpy as np
from sentence_transformers import SentenceTransformer

emb = SentenceTransformer('all-MiniLM-L6-v2')
client = anthropic.Anthropic()

def chunk(text: str, size: int = 120, overlap: int = 20) → list[str]:
    words = text.split()
    return [' '.join(words[i:i + size]) for i in range(0, len(words), size - overlap)]

docs = {'leave_policy.txt': open('leave_policy.txt', encoding='utf-8').read()}
chunks = [(src, c) for src, t in docs.items() for c in chunk(t)]
vecs = emb.encode([c for _, c in chunks], normalize_embeddings=True)

def answer(question: str, k: int = 4) → str:
    q = emb.encode([question], normalize_embeddings=True)[0]
    top = np.argsort(vecs @ q)[::-1][:k]
    context = '\n'.join(f"<doc id='{i}' source='{chunks[i][0]}'>{chunks[i][1]}</doc>" for i in top)
    msg = client.messages.create(
        model='claude-opus-5-5', max_tokens=1500,
        system='Answer only from the documents. Cite doc ids like [doc 3]. If the answer is not in
```

```
them, say you do not know.',
        messages=[{'role': 'user', 'content': f'<documents>{context}</documents>\n<question>{question}
</question>'}],
    )
    return ''.join(b.text for b in msg.content if b.type == 'text')

print(answer('How many days of paternity leave do employees get?'))
```

COMMON MISTAKES

- Chunks too large or too small; tune with real questions.
- No 'I don't know' instruction, so the model invents answers when retrieval fails.
- Ignoring document permissions, so users see content they should not.
- Never evaluating retrieval separately from generation.

INTERVIEW QUESTIONS AND ANSWERS

Q. Explain RAG end to end.

A. Documents are parsed, chunked, embedded and stored with metadata. At query time the question is embedded, the nearest chunks are retrieved (often with filtering and reranking), and the model answers using only those chunks with citations. Retrieval and answer quality are evaluated separately.

Q. RAG gives wrong answers. How do you debug it?

A. Check whether the right chunks were retrieved. If not, fix parsing, chunking, embeddings, filters or add hybrid search/reranking. If yes, fix the prompt (grounding, citation rules) or model. Add failing cases to the test set.

Q. When would you not use RAG?

A. When the task needs behaviour or style change rather than facts (fine-tuning or prompting), when all needed data fits easily in the prompt, or when exact structured queries against a database are better.

HANDS-ON EXERCISE

> Build a RAG assistant over a company's HR policies with citations and a 30-question evaluation, and report retrieval hit rate and answer faithfulness.

OFFICIAL AND RECOMMENDED RESOURCES

- Pinecone: build a RAG chatbot https://docs.pinecone.io/guides/get-started/build-a-rag-chatbot
- Pinecone: chunking strategies https://www.pinecone.io/learn/chunking-strategies/
- Anthropic: contextual retrieval https://www.anthropic.com/news/contextual-retrieval
- Claude: citations https://docs.claude.com/en/docs/build-with-claude/citations